Christopher Le

CSC 6850 – Machine Learning

Spring 2026

Introduction

OCSR or Optical Chemical Structure Recognition is the translation of chemical structure images into machine readable formats such as SMILES or molecular graphs. These formats preserve chemical information such as rings, branches, atoms, and bonds. This project compares different deep learning models for OCSR.

This project focuses on converting chemical structure images into SMILES strings. Models must be able to read a structure and generate the correct sequence of SMILES tokens. Any deviation from the correct sequence can give a completely different model. For instance, one deviation could be the difference between oxygen gas and ozone.

The original goal was to compare a CNN-based model, a Transformer-based model, and a graph-based model. However, the project was too large to fully implement. This was narrowed down involves using three deep learning models where CNN is used to encode and RNN, GRU, and LSTM is used to decode sequences.

Related Works

This project draws inspiration from two main OCSR papers.

The first is SwinOCSR by Xu et al., “SwinOCSR: End-to-end optical chemical structure recognition using a Swin Transformer,” Journal of Cheminformatics, 2022. This paper treats OCSR as an image to sequencing problem.

The second is Oldenhof, De Brouwer, Arany, and Moreau, “Atom-level optical chemical structure recognition with limited supervision,” CVPR, 2024. This paper focuses more on atom-recognition and graph reconstruction.

Some external sources I used were PubChem, Decimer, and EPA Comptox. Pubchem is a database of chemical with SMILES names, this data was used to train and test models. EPA CompTox is a separate database from PubChem and could be used to

test versus PubChem. Decimer is a database of hand-drawn chemical structures which could be used as a test set as well.

My project is simpler than these full OCSR systems. SwinOCSR or atom level graph reconstruction was not reproduced but the direction was similar.

Methodology

The final project uses an image-to-sequence model. The input is a chemical structure image, and the output is a SMILES string. When the model gets a chemical image, it goes through a CNN encoder where it extracts visual features from the molecular image. The recurrent decoder uses those features to generate a SMILES sequence one token at a time.

Models

I compared three recurrent decoder models. They include CNN-RNN, CNN-GRU, CNN-LSTM. All models used CNN for the encoder but varied with the decoder. The CNN encoder is used because the input is an image. Chemical structures consist of lines, corners, letters, rings, and branches. Models can learn from these features and learn features such as single bonds, double bonds, triple bonds, atom labels, rings, and branches. The CNN converts the image into a compact feature vector and the vecto is then passed into the decoder.

The CNN-RNN model is the simplest model. Each token is processed one at a time and it keeps information from the previous token. However, when compared to the other two decoders, RNN has a weaker memory compared to GRU and LSTM. When sequences become extremely long, it begins to struggle. The CNN-RNN model serves as a baseline for comparisons.

The CNN-GRU model uses a Gated Recurrent Unit as the decoder. A GRU is an improved version of RNN where it uses gates to control what information should be remembered (update gate) and what information should be forgotten (reset gate).

The CNN-LSTM or Long Short-Term Memory has more memory control than RNN or GRU. It has an input gate, forget gate, output gate, hidden state, and cell state.

Smiles Tokenization

Before supervised training the models, SMILES strings need to be tokenized. The tokenizer splits the string by character but also keeps certain combination of symbols together such as C and l (chlorine) and bracketed atoms. The tokenizer also adds tokens for the start and end of tokens, padding tokens (for short sequences), and unknown tokens (where the symbol is unknown).

Implementation

The project was implemented in Python using Pytorch. It was used for model implementation, CNN encoding, loss calculation, and optimization. It was also used for RNN, GRU, and LSTM decoders. For chemical processing, RDKIT was used. Examples of its use included validating SMILES strings, generating images, canonicalizing SMILES strings, and checking whether generated strings were valid.

The model was trained using Adam optimization and Cross Entropy Loss. Padding tokens are ignored during loss calculation so models could not be rewarded or punished for predicting padded sequence positions.

Datasets

PubChem was the first data set and the main training dataset. PubChem provides chemical structures and SMILES string. Molecule images were generated from SMILES strings using RDKit. The dataset consist of 400 molecule images split 80-20 for training and testing. 320 training images and 80 testing images were created. This was done because images will be standardized by RDKit.

The second dataset comes from EPA CompTox. The reason for this is to see whether a model trained on PubChem could be used on models from a different database. For EPA, I collected chemical naems and SMILES strings and generated molecule images using RDKit in a similar fashion to PubChem. It is still structured data but from a different source.

The third dataset was the DECIMER hand-drawn molecule dataset. This data set is less structured when compared to PubChem and EPA because the images are hand drawn. Lines are not as clean or uniform so structures can vary visually. I used a limited DECIMER subset for evaluation. Preprocessing made sure the images and their corresponding SMILES string was valid. This dataset was included to see how the model would perform with changes in visual clarity.

Experiment

The three models were compared and with each two parameters were varied. The two parameters are learning rate and hidden dimensions. The learning rate included 0.001, 0.0005, and 0.0001. Hidden dimensions included 128, 256, and 512. The embedding dimension was kept at 128 and models were trained for 10 epochs.

Models were tested for token accuracy, exact SMILES match, canonical SMILES match, and valid SMILES rate. Token accuracy measured how many predicted tokens match the correct target tokens. This is useful because it shows whether the model is learning local SMILES pattern. Exact SMILES match measures whether the entire string matches. Canonical SMILES uses RDKit to check if predicted molecule is equivalent (as some molecules can be named differently based on where you start on the graph). Valid SMILES rate measures whether RDKIT can parse the SMILES string.

Results

The table below summarizes the results from training and testing PubChem.

Dataset	Model	Learning Rate	Hidden Dim	Token Accuracy	Exact Accuracy	Canonical Match	Valid SMILES
pubchem	rnn	0.001	128	57.38%	0.00%	0.00%	100.00%
pubchem	gru	0.001	128	57.55%	0.00%	0.00%	100.00%
pubchem	lstm	0.001	128	59.67%	0.00%	0.00%	100.00%
pubchem	rnn	0.001	256	62.17%	0.00%	0.00%	100.00%
pubchem	gru	0.001	256	64.00%	0.00%	0.00%	100.00%
pubchem	lstm	0.001	256	62.17%	0.00%	0.00%	100.00%
pubchem	rnn	0.001	512	64.96%	0.00%	0.00%	100.00%
pubchem	gru	0.001	512	68.42%	0.00%	0.00%	100.00%
pubchem	lstm	0.001	512	66.79%	0.00%	0.00%	100.00%
pubchem	rnn	0.0005	128	54.39%	0.00%	0.00%	100.00%
pubchem	gru	0.0005	128	53.09%	0.00%	0.00%	100.00%
pubchem	lstm	0.0005	128	55.35%	0.00%	0.00%	100.00%
pubchem	rnn	0.0005	256	59.71%	0.00%	0.00%	100.00%
pubchem	gru	0.0005	256	59.11%	0.00%	0.00%	100.00%
pubchem	lstm	0.0005	256	57.68%	0.00%	0.00%	100.00%
pubchem	rnn	0.0005	512	62.50%	0.00%	0.00%	100.00%
pubchem	gru	0.0005	512	63.10%	0.00%	0.00%	100.00%

pubchem	lstm	0.0005	512	64.23%	0.00%	0.00%	100.00%
pubchem	rnn	0.0001	128	46.78%	0.00%	0.00%	100.00%
pubchem	gru	0.0001	128	44.12%	0.00%	0.00%	100.00%
pubchem	lstm	0.0001	128	43.62%	0.00%	0.00%	100.00%
pubchem	rnn	0.0001	256	50.43%	0.00%	0.00%	100.00%
pubchem	gru	0.0001	256	50.20%	0.00%	0.00%	100.00%
pubchem	lstm	0.0001	256	48.60%	0.00%	0.00%	100.00%
pubchem	rnn	0.0001	512	54.06%	0.00%	0.00%	100.00%
pubchem	gru	0.0001	512	51.60%	0.00%	0.00%	100.00%
pubchem	lstm	0.0001	512	51.33%	0.00%	0.00%	100.00%

When looking at learning rate, the learning rate of 0.001 was best because it updated weights enough given the amount of training epochs (10). The learning rate of 0.0005 worked, but it gave a lower accuracy.

The best results came from the hidden dimension of 512. This suggests that the recurrent decoder needed more capacity to model SMILES token patterns. The smaller dimensions could not represent longer/complex sequences.

The table below summarizes the main results across the PubChem test split and external EPA and DECIMER datasets at the learning rate and hidden dimension with the highest token accuracy.

Dataset	Model	Learning Rate	Hidden Dimension	Embedding Dimension	Token Accuracy	Exact Match	Canonical Match	Valid SMILES Rate
PubChem	CNN-RNN	0.001	512	128	64.96%	0.00%	0.00%	100.00%
PubChem	CNN-GRU	0.001	512	128	68.42%	0.00%	0.00%	100.00%
PubChem	CNN-LSTM	0.001	512	128	66.79%	0.00%	0.00%	100.00%
EPA CompTox	CNN-RNN	0.001	512	128	40.74%	0.00%	0.00%	100.00%
EPA CompTox	CNN-GRU	0.001	512	128	41.14%	0.00%	0.00%	100.00%
EPA CompTox	CNN-LSTM	0.001	512	128	39.85%	0.00%	0.00%	100.00%
DECIMER	CNN-RNN	0.001	512	128	37.49%	0.00%	0.00%	100.00%
DECIMER	CNN-GRU	0.001	512	128	38.62%	0.00%	0.00%	100.00%

Dataset	Model	Learning Rate	Hidden Dimension	Embedding Dimension	Token Accuracy	Exact Match	Canonical Match	Valid SMILES Rate
DECIMER	CNN-LSTM	0.001	512	128	37.49%	0.00%	0.00%	100.00%

The best PubChem result came from CNN-GRU with the learning rate of 0.001, hidden dimension 512, and embedding dimension 128. This model reached 68.42% train token accuracy. However, all exact match and canonical match scores were 0%. This means that even when the model predicted many individual SMILES token correctly, it still did not generate a complete molecule string that matched the target.

The external datasets were harder. EPA CompTox token accuracy was around 40%, and DECIMER token accuracy was around 38%. This drop is expected due to different sources of images. The DECIMER accuracy drop was also expected due to molecules being hand drawn.

When comparing the three models together, across the hyperparameter grid, the GRU had the single best result and LSTM had the second single best result. On average, GRU had higher token accuracies compared to both LSTM and RNN.

Analysis

The main result is that models learned partial SMILES patterns but did not solve complete OCSR. The best model reached 68.42% token accuracy on PubChem, but exact match and canonical match were still 0%.

One important failure case was the repeated carbon-carbon chain prediction (CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC). This is a valid string and exists as a long chain of carbons. One reason the model may have gravitated toward long carbon-chain predictions is that C is a common token in SMILES strings. Many organic compounds contain carbon. Even though the full model may be wrong, it increases the token accuracy by repeatedly choosing C.

This is likely due to the fact that instead of fully understanding the image structure, it leaned towards common SMILES tokens that were likely to appear in training data. The model reduces token loss by predicting carbon repeatedly which would explain the why the token accuracy increased to 68% but the exact match and canonical match stayed at 0. The model failed match the image exactly.

The GRU model had the best results and this may be the fact that LSTM was too complex. The dataset may have not been complex enough for LSTM to perform well. That said, the biggest limitation was the dataset size. The project used hundreds of images but it could use more data to recognize a myriad of different structures.

Conclusion

The project started as a comparison of CNN, Transformers, and graph-based OCSR models, but after feedback, the project was narrowed down to a CNN recurrent decoder project. The final models were CNN-RNN, CNN-GRU, and CNN-LSTM with the best model being CNN_GRU. Even though this is the case, exact matches were never reached and full molecule reconstruction was not obtained. Limitations include needing more data, stronger decoding or better chemicals.

Future implementations of this project could focus on different things such as training on a larger PubChem dataset, or a different algorithm for selecting the best model. Other representations of chemical compounds could be explored and other encoders could be explored as well.

Works Cited

1. Xu, X. et al. “SwinOCSR: End-to-end optical chemical structure recognition using a Swin Transformer.” Journal of Cheminformatics, 2022.
2. Oldenhof, M., De Brouwer, E., Arany, A., and Moreau, Y. “Atom-level optical chemical structure recognition with limited supervision.” CVPR, 2024.
3. DECIMER Hand-drawn Molecule Dataset. Zenodo record 7617107.
4. RDKit: Open-source cheminformatics toolkit, used for SMILES validation, canonicalization, and molecule image rendering.
5. PyTorch and torchvision: deep learning framework and image transformation utilities used for model implementation.